

本节内容

哈夫曼树

知识总览

哈夫曼树

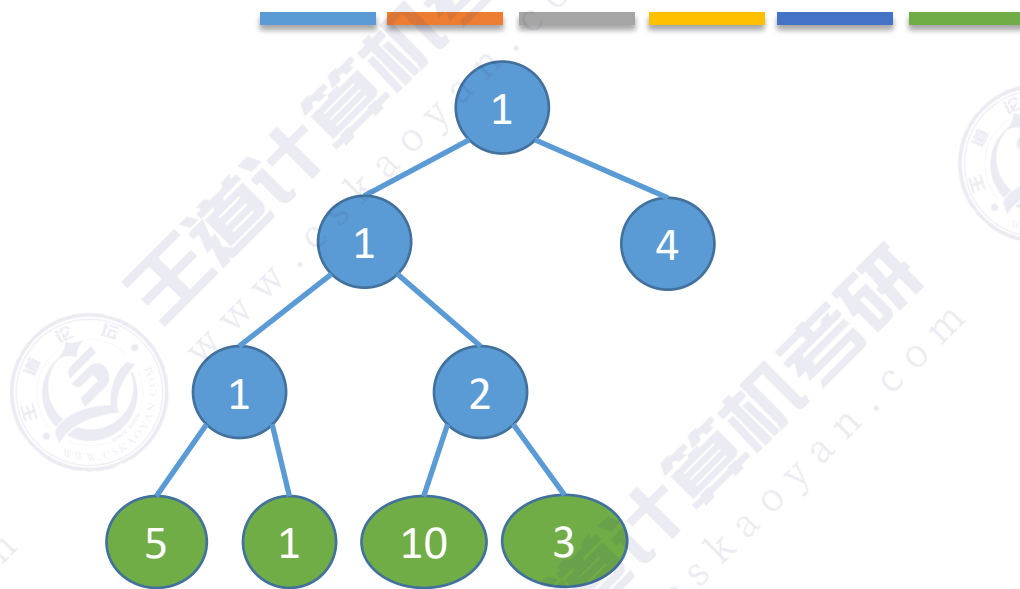
带权路径长度

哈夫曼树的定义

哈夫曼树的构造

哈夫曼编码

带权路径长度



结点的**权**：有某种现实含义的数值（如：表示结点的重要性等）

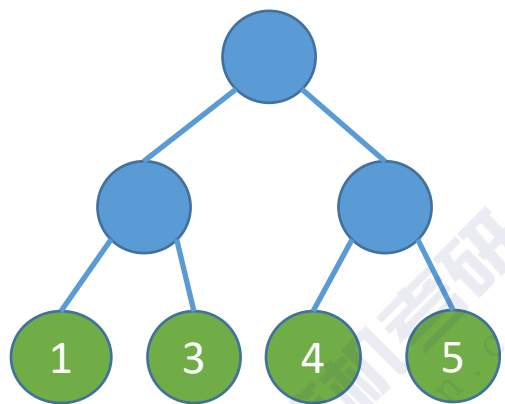
结点的**带权路径长度**：从树的根到该结点的路径长度（经过的边数）与该结点上权值的乘积

树的**带权路径长度**：树中所有**叶结点**的带权路径长度之和（WPL, Weighted Path Length）

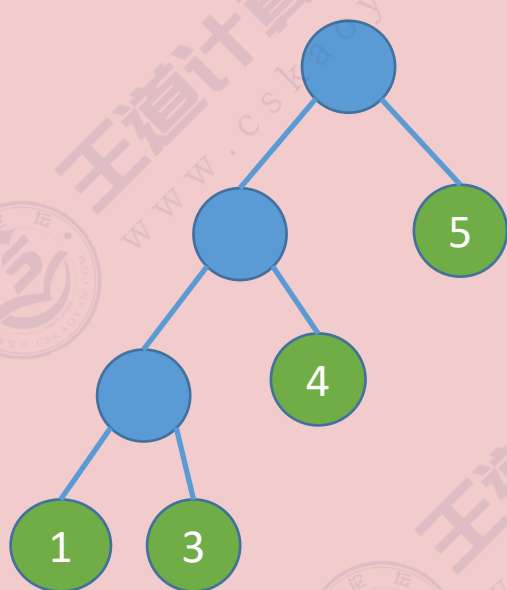
$$WPL = \sum_{i=1}^n w_i l_i$$

哈夫曼树的定义

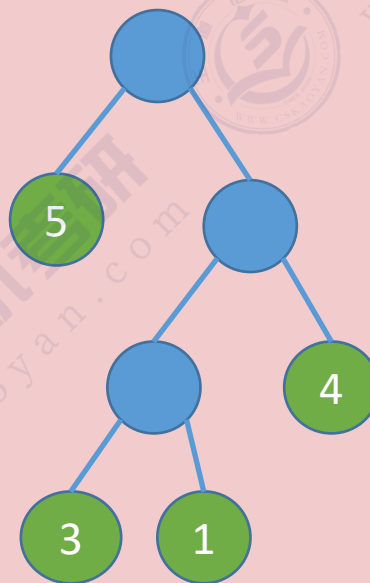
都是哈夫曼树



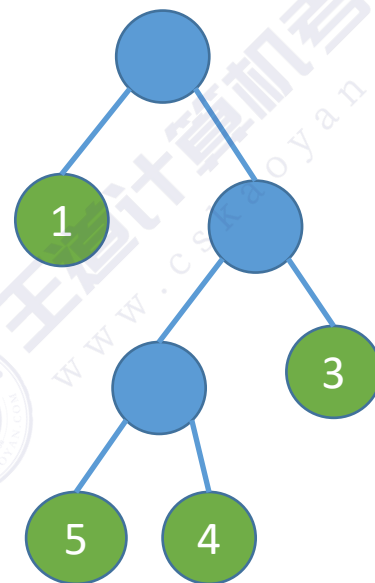
$$WPL = 2 \times 1 + 2 \times 3 + 2 \times 4 + 2 \times 5 = 26$$



$$WPL = 1 \times 5 + 2 \times 4 + 3 \times 1 + 3 \times 3 = 25$$



$$WPL = 1 \times 5 + 2 \times 4 + 3 \times 1 + 3 \times 3 = 25$$



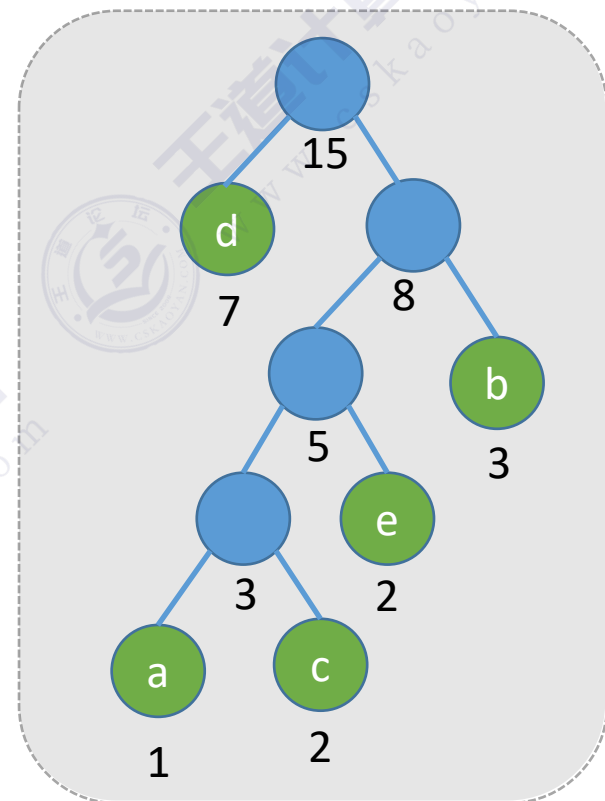
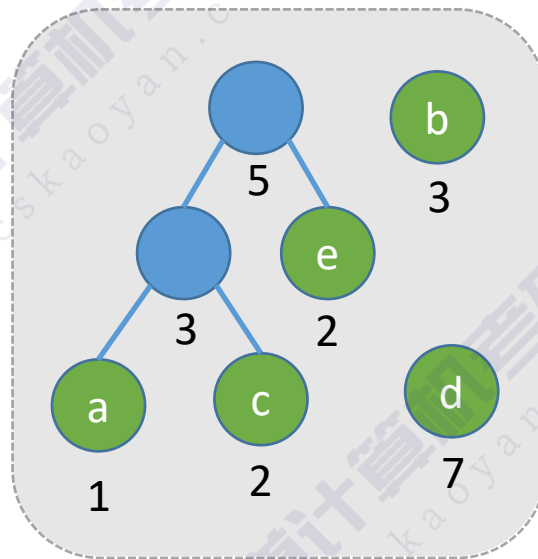
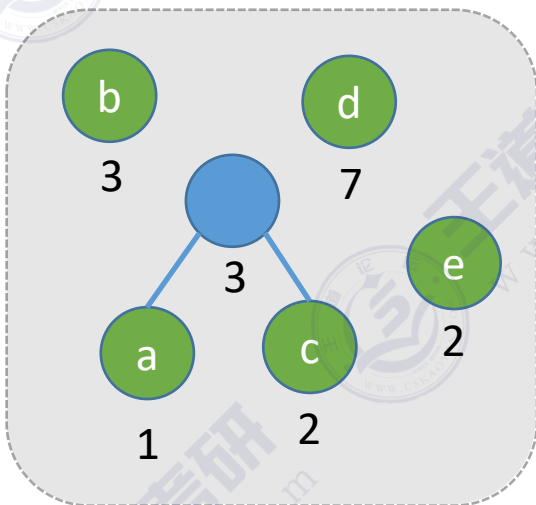
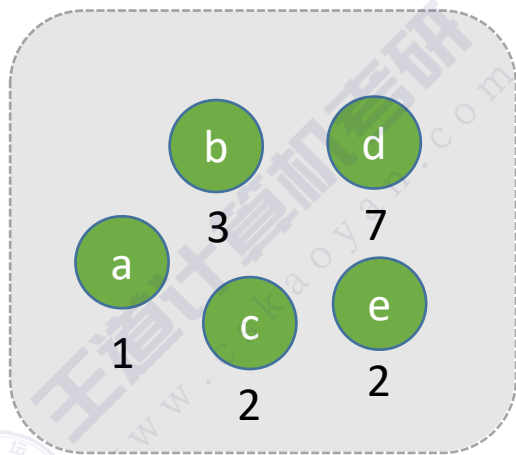
$$WPL = 1 \times 1 + 2 \times 3 + 3 \times 5 + 3 \times 4 = 34$$

在含有 n 个带权叶结点的二叉树中，其中带权路径长度（WPL）最小的二叉树称为哈夫曼树，也称最优二叉树

哈夫曼树的构造

给定 n 个权值分别为 $w_1, w_2, \dots, w_n$ 的结点，构造哈夫曼树的算法描述如下：

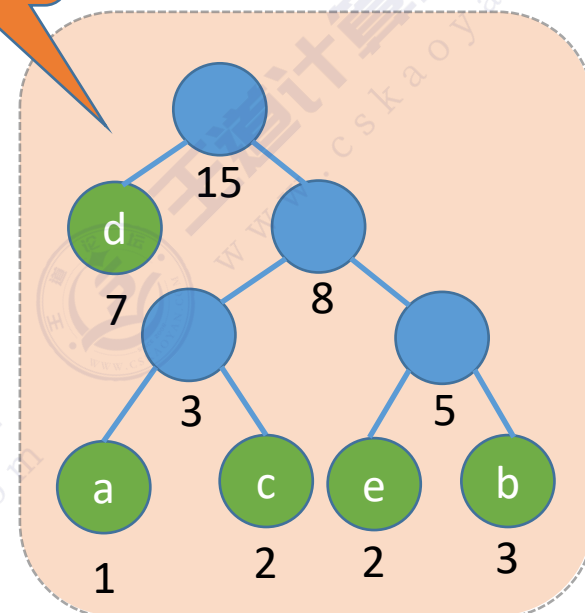
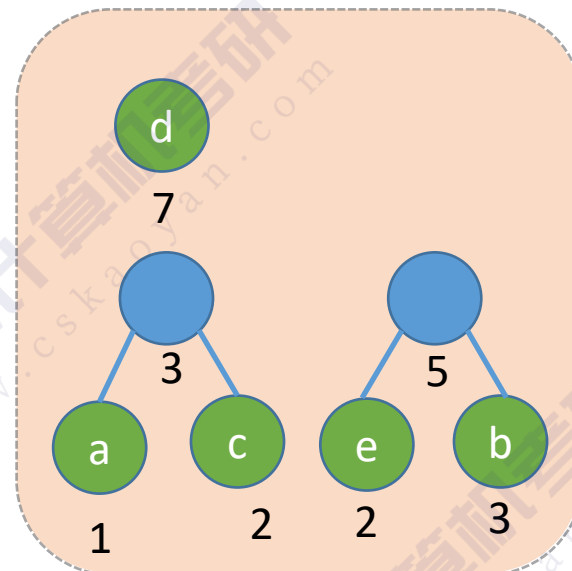
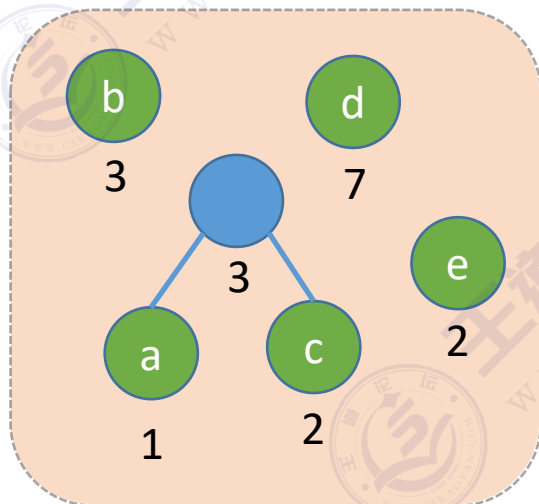
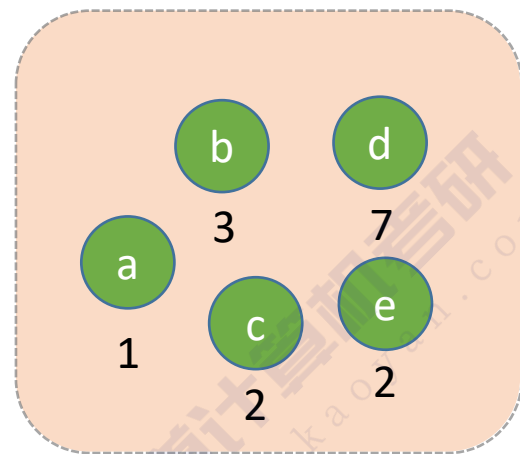
- 1) 将这 n 个结点分别作为 n 棵仅含一个结点的二叉树，构成森林 F 。
- 2) 构造一个新结点，从 F 中选取两棵根结点权值最小的树作为新结点的左、右子树，并且将新结点的权值置为左、右子树上根结点的权值之和。
- 3) 从 F 中删除刚才选出的两棵树，同时将新得到的树加入 F 中。
- 4) 重复步骤2) 和3)，直至 F 中只剩下一棵树为止。



- 1) 每个初始结点最终都成为叶结点，且权值越小的结点到根结点的路径长度越大
- 2) 哈夫曼树的结点总数为 $2n - 1$
- 3) 哈夫曼树中不存在度为1的结点。
- 4) 哈夫曼树并不唯一，但WPL必然相同且为最优

$$WPL_{\min} = 1 \times 7 + 2 \times 3 + 3 \times 2 + 4 \times 1 + 4 \times 2 = 31$$

哈夫曼树的构造



左右顺序任意

$$WPL=1*7+3*(1+2+2+3)=31$$

哈夫曼编码



电报——点、划 两个信号（二进制0/1）

哈夫曼编码

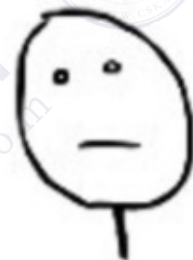
固定长度编码——每个字符用相等长度的二进制位表示

A——0100 0001
B——0100 0010
C——0100 0011
D——0100 0100

ASCII编码

A——00
B——01
C——10
D——11

每个字符用长度为2的二进制表示

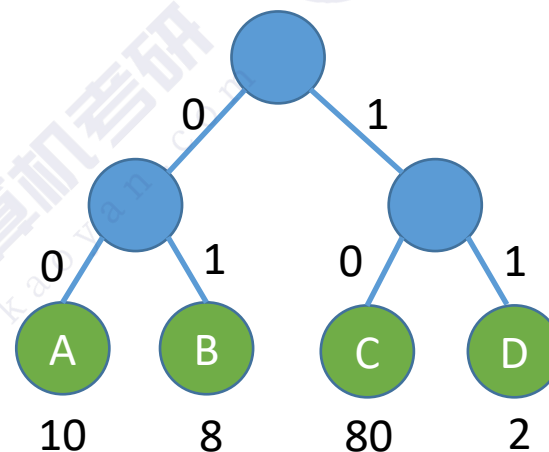


小渣

100个选择题



老渣



假设，100题中有80题选C，10题选A，8题选B，2题选D

所有答案的二进制长度=80*2+10*2+8*2+2*2=200 bit

$$WPL = 80 \times 2 + 10 \times 2 + 8 \times 2 + 2 \times 2 = 200$$

哈夫曼编码

固定长度编码——每个字符用相等长度的二进制位表示

A——00
B——01
C——10
D——11

每个字符用长度为2的二进制表示



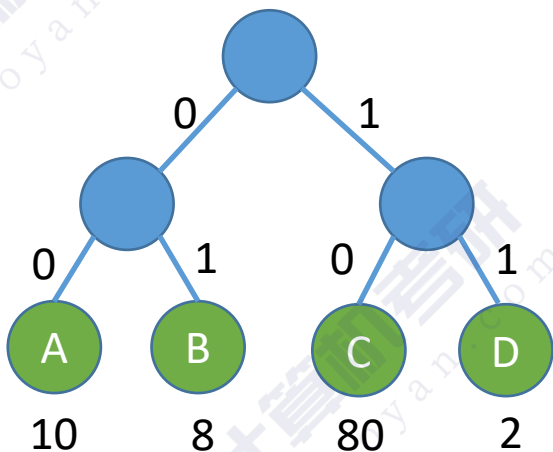
小渣

100个选择题



老渣

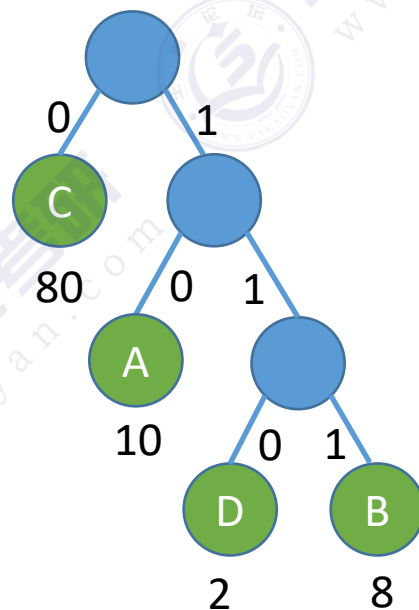
假设，100题中有80题选C，10题选A，8题选B，2题选D
所有答案的二进制长度= $80*2+10*2+8*2+2*2=200$ bit



$$WPL = 80*2 + 10*2 + 8*2 + 2*2 = 200$$

C——0
A——10
B——111
D——110

可变长度编码——允许对不同字符用不等长的二进制位表示



$$WPL = 80*1 + 10*2 + 2*3 + 8*3 = 130$$

哈夫曼编码

可变长度编码——允许对不同字符用不等长的二进制位表示

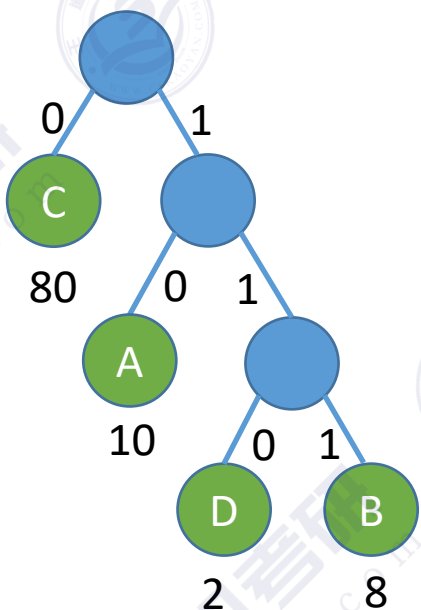
若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码

好的收到，CBBD



前缀码解
码无歧义

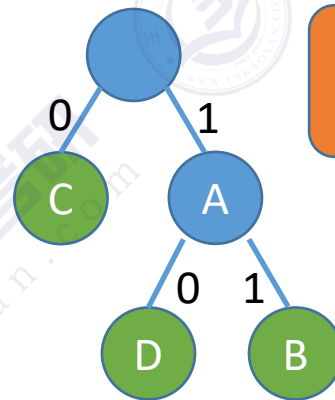
C—0
A—10
B—111
D—110



$$WPL = 80 \times 1 + 10 \times 2 + 2 \times 3 + 8 \times 3 = 130$$

CAAABD: 0101010111110

C—0
A—1
B—111
D—110



非前缀码解
码有歧义



CAAABD: 0111111110

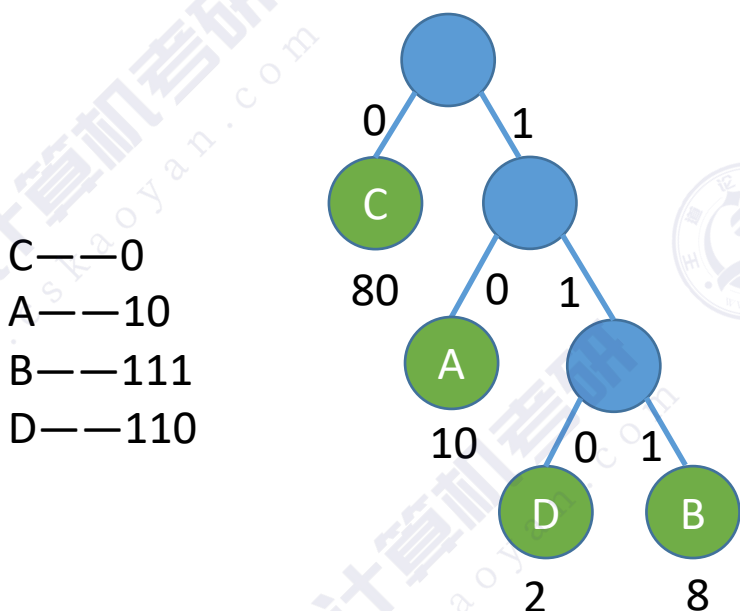
哈夫曼编码

固定长度编码——每个字符用相等长度的二进制位表示

可变长度编码——允许对不同字符用不等长的二进制位表示

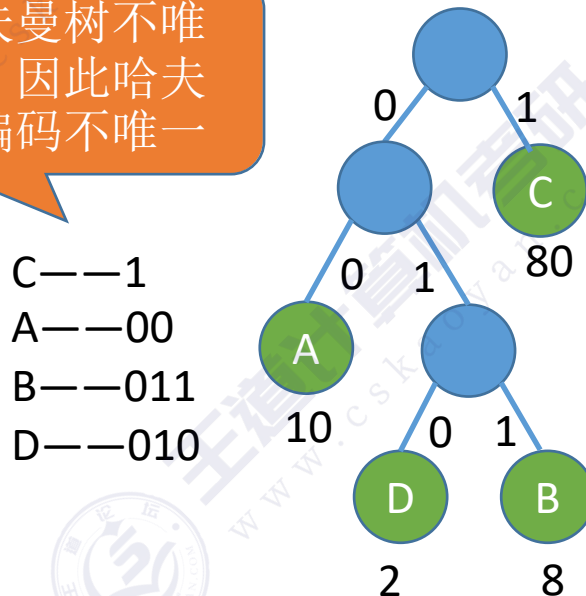
若没有一个编码是另一个编码的前缀，则称这样的编码为前缀编码

有哈夫曼树得到哈夫曼编码——字符集中的每个字符作为一个叶子结点，各个字符出现的频度作为结点的权值，根据之前介绍的方法构造哈夫曼树



$$WPL = 80 \times 1 + 10 \times 2 + 2 \times 3 + 8 \times 3 = 130$$

哈夫曼树不唯一，因此哈夫曼编码不唯一



$$WPL = 80 \times 1 + 10 \times 2 + 2 \times 3 + 8 \times 3 = 130$$

哈夫曼编码可用于数据压缩



英文字母频次

英文字母使用频率表:(%)

A 8.19	B 1.47	C 3.83	D 3.91	E 12.25	F 2.26	G 1.71
H 4.57	I 7.10	J 0.14	K 0.41	L 3.77	M 3.34	N 7.06
O 7.26	P 2.89	Q 0.09	R 6.85	S 6.36	T 9.41	
U 2.58	V 1.09	W 1.59	X 0.21	Y 1.58	Z 0.08	

试试设计哈夫曼编码，并计算数据压缩率

知识回顾与重要考点

哈夫曼树

概念

结点的权：某种特定含义的数值

结点的带权路径长度 = 根到结点路径长度 * 结点的权值

树的带权路径长度(WPL) = 树中所有叶子结点的带权路径长度之和

哈夫曼树（最优二叉树）：在含有给定的n个带权叶结点的二叉树中，WPL 最小的二叉树

构造哈夫曼树

每次选两个根节点权值最小的树合并，并将二者权值之和作为新的根节点的权值

哈夫曼树不唯一，但WPL必然都是最小值

将字符频次作为字符结点权值，构造哈夫曼树，即可得哈夫曼编码，可用于数据压缩

哈夫曼编码

前缀编码——没有一个编码是另一个编码的前缀

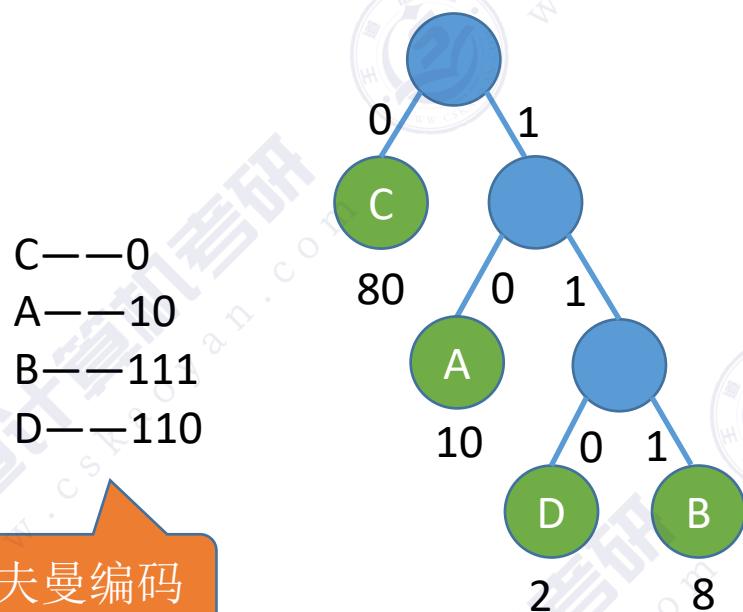
固定长度编码——每个字符用相等长度的二进制位表示

可变长度编码——允许对不同字符用不等长的二进制位表示

结局

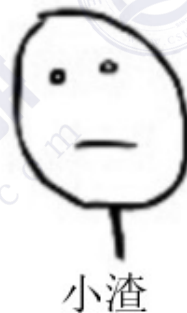
可变长度编码——允许对不同字符用不等长的二进制位表示

若没有一个编码是另一个编码的前缀，则称这样的编码为**前缀编码**



哈夫曼编码

$$WPL = 80 \times 1 + 10 \times 2 + 2 \times 3 + 8 \times 3 = 130 \text{ bit}$$



100道选择题



本节内容

并查集

知识总览

并查集

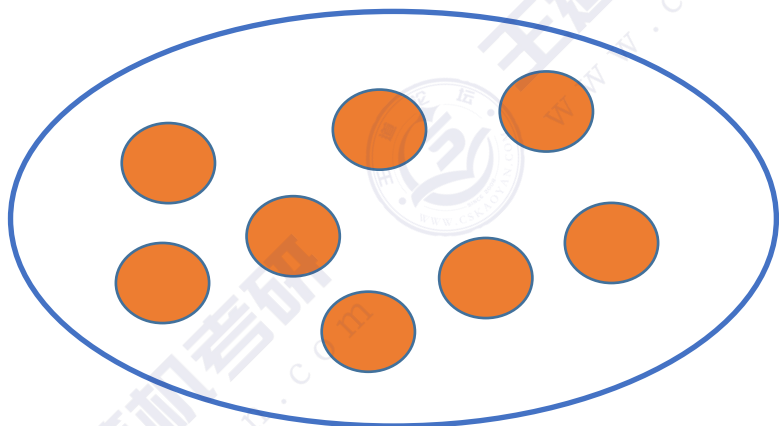
如何表示"集合"关系?

"并查集"的代码实现

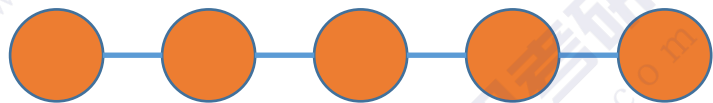
"并查集"的优化

漏网之鱼：逻辑结构——“集合”

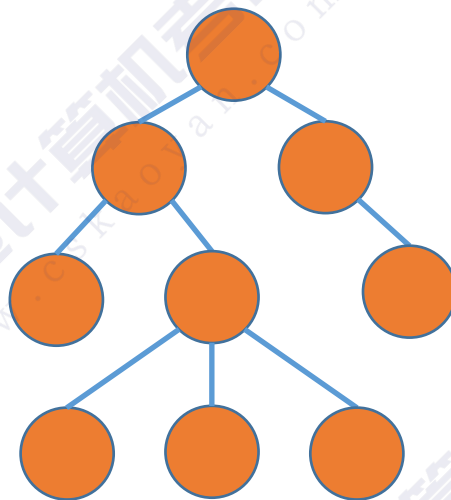
逻辑结构——数据元素之间的逻辑关系是什么？



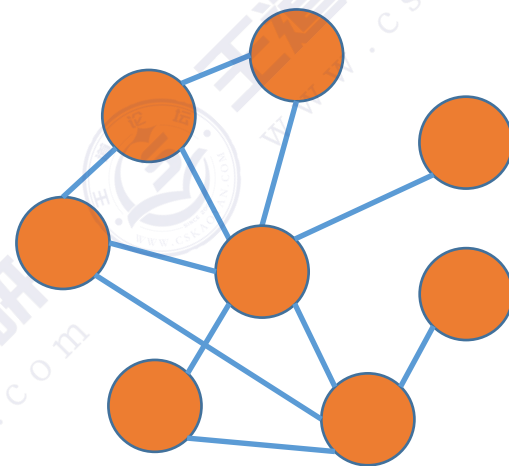
集合



线性结构

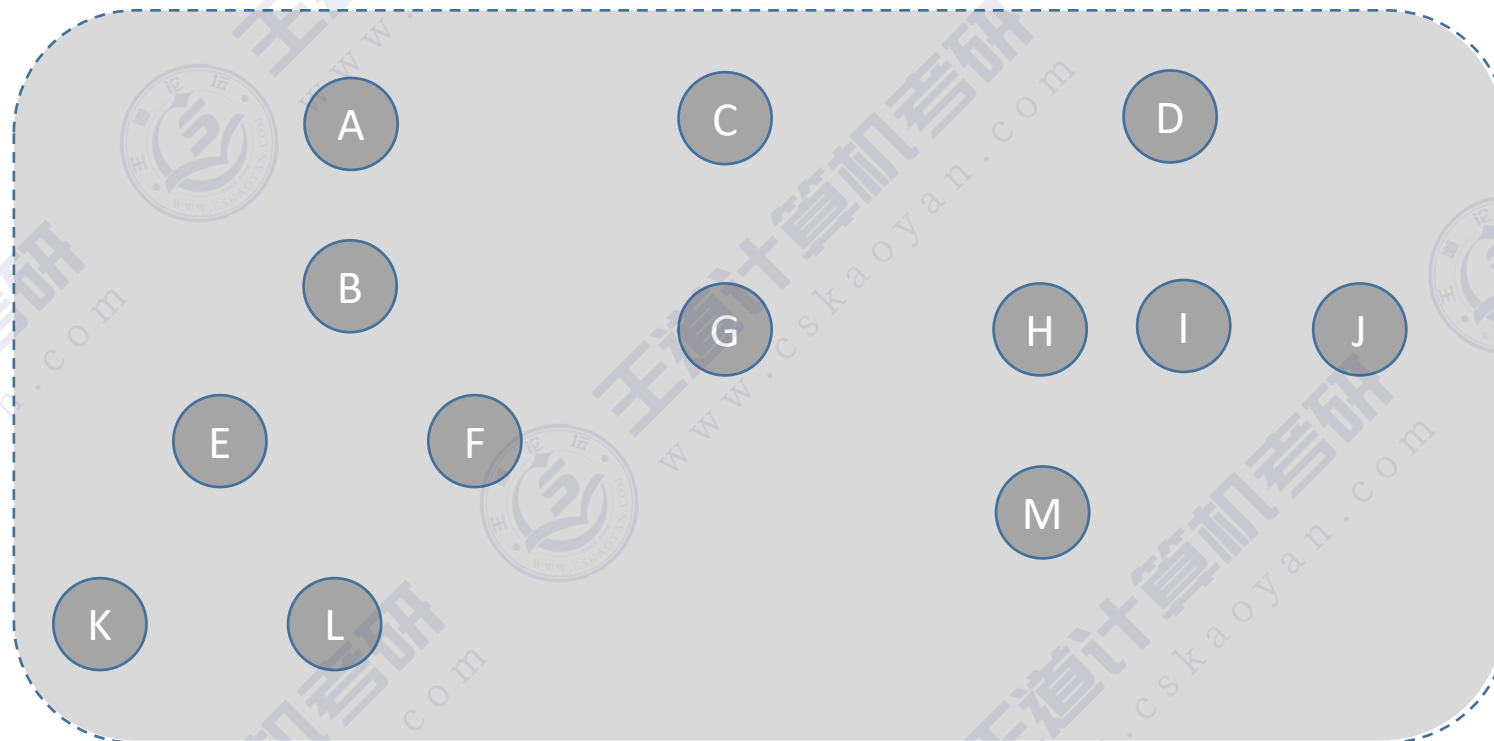


树形结构



图结构

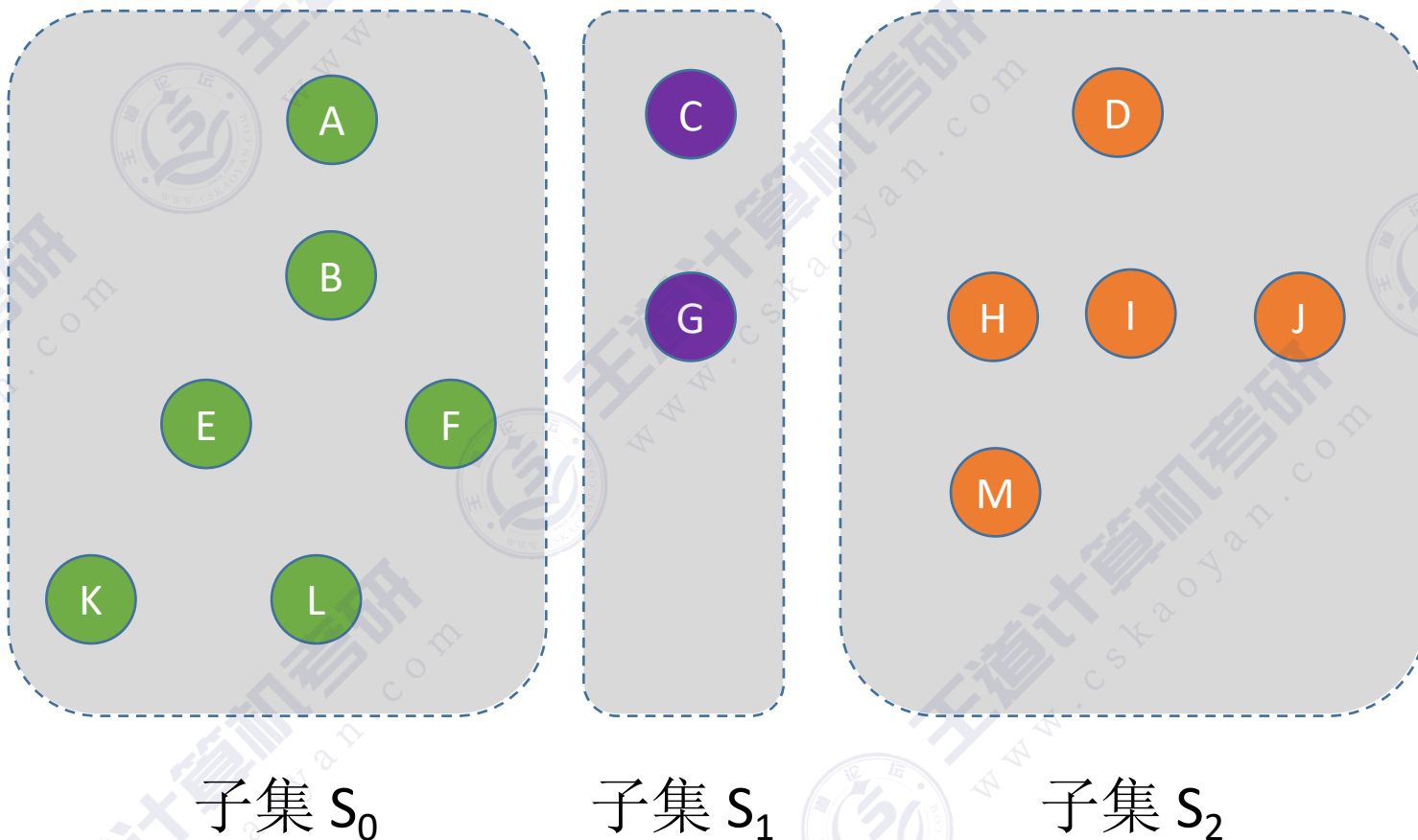
逻辑结构——“集合”



所有元素的全集 S

逻辑结构——“集合”

将各个元素划分为若干个互不相交的子集

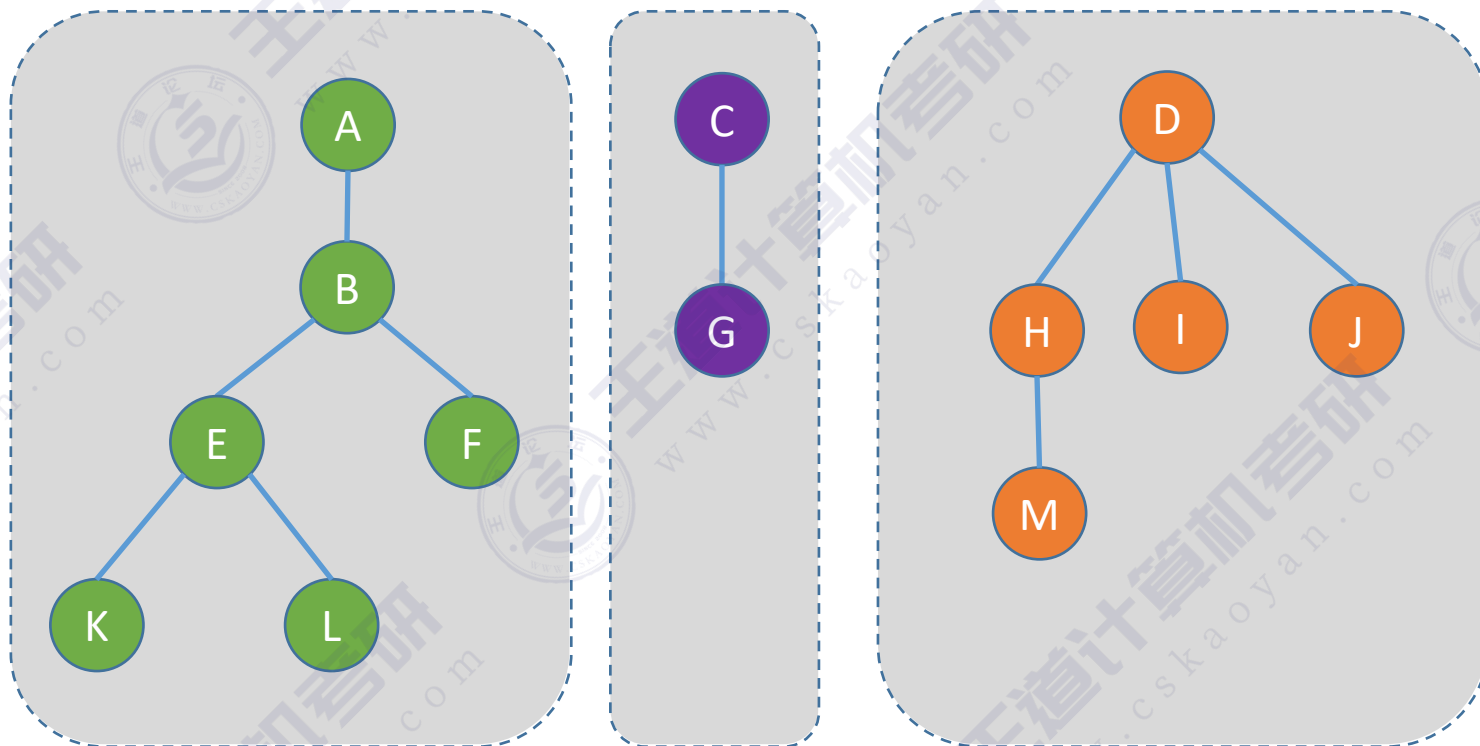


怎么用代码表示这种逻辑关系???



回顾：森林

森林。森林是 m ($m \geq 0$) 棵互不相交的树的集合



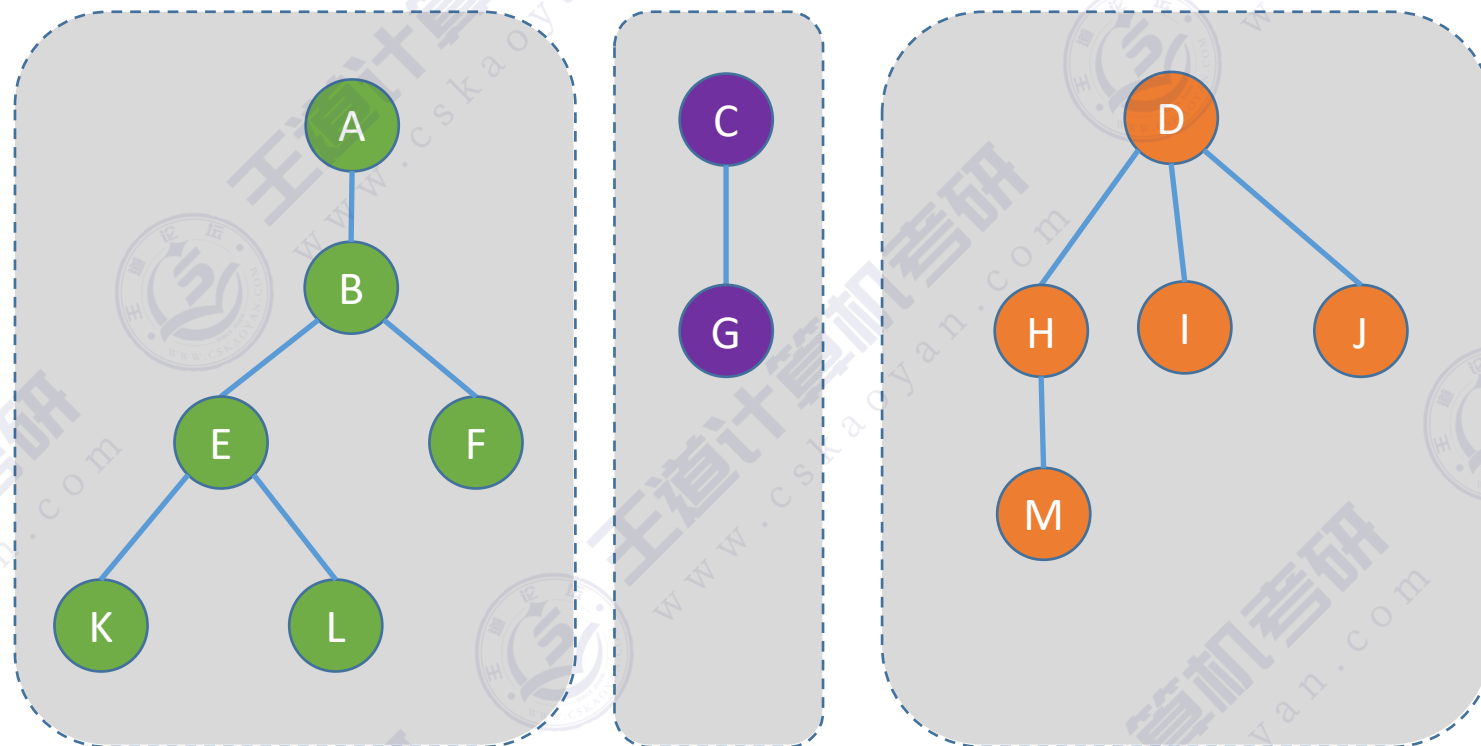
同一子集中的各个元素，组织成一棵树



灵稽一动

三棵树组成的森林

用互不相交的树，表示多个“集合”



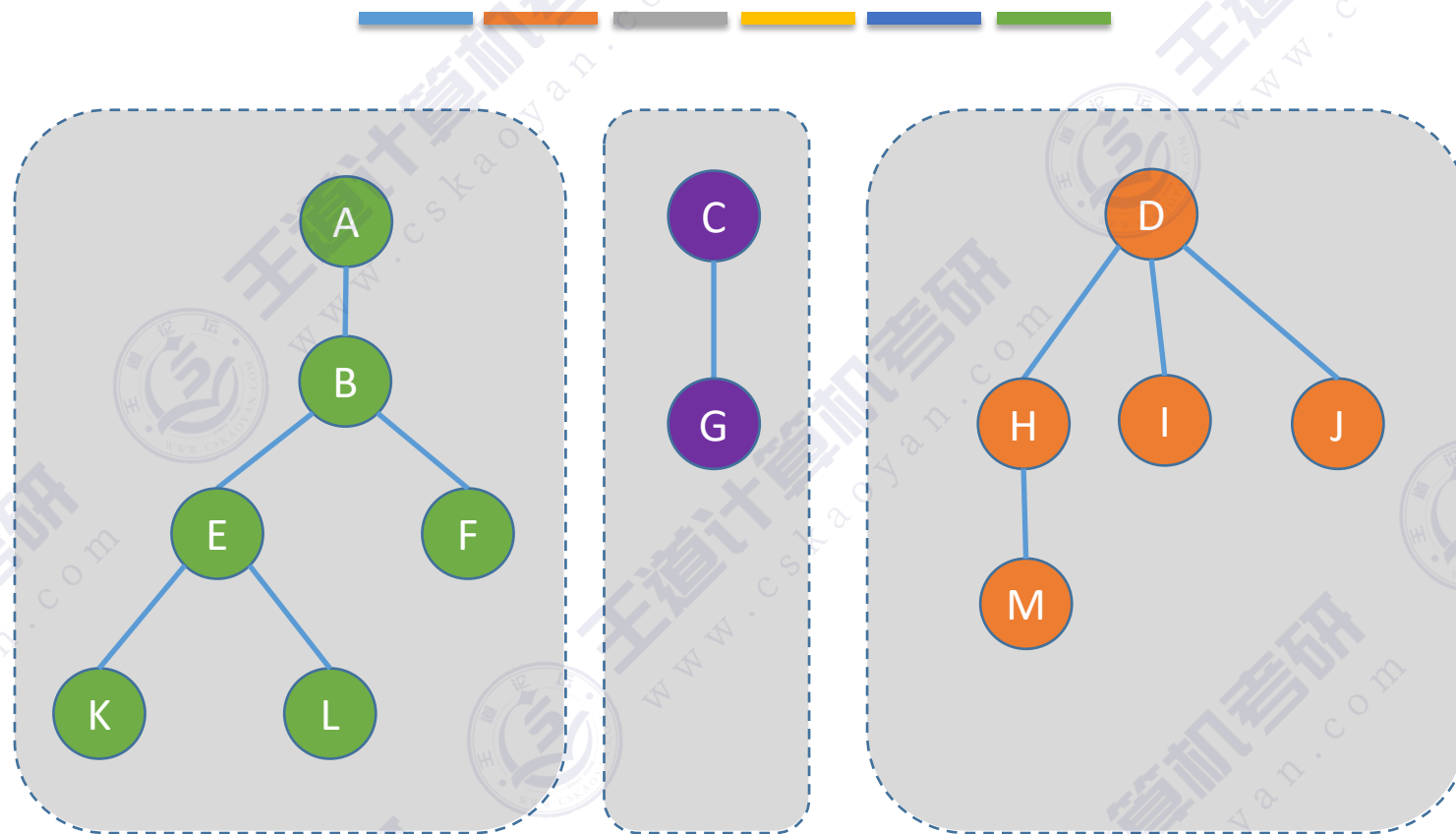
如何“查”到一个元素到底属于哪一个集合？
—— 从指定元素出发，一路向北，找到根节点

如何判断两个元素是否属于同一个集合？
—— 分别查到两个元素的根，判断根节点是否相同即可



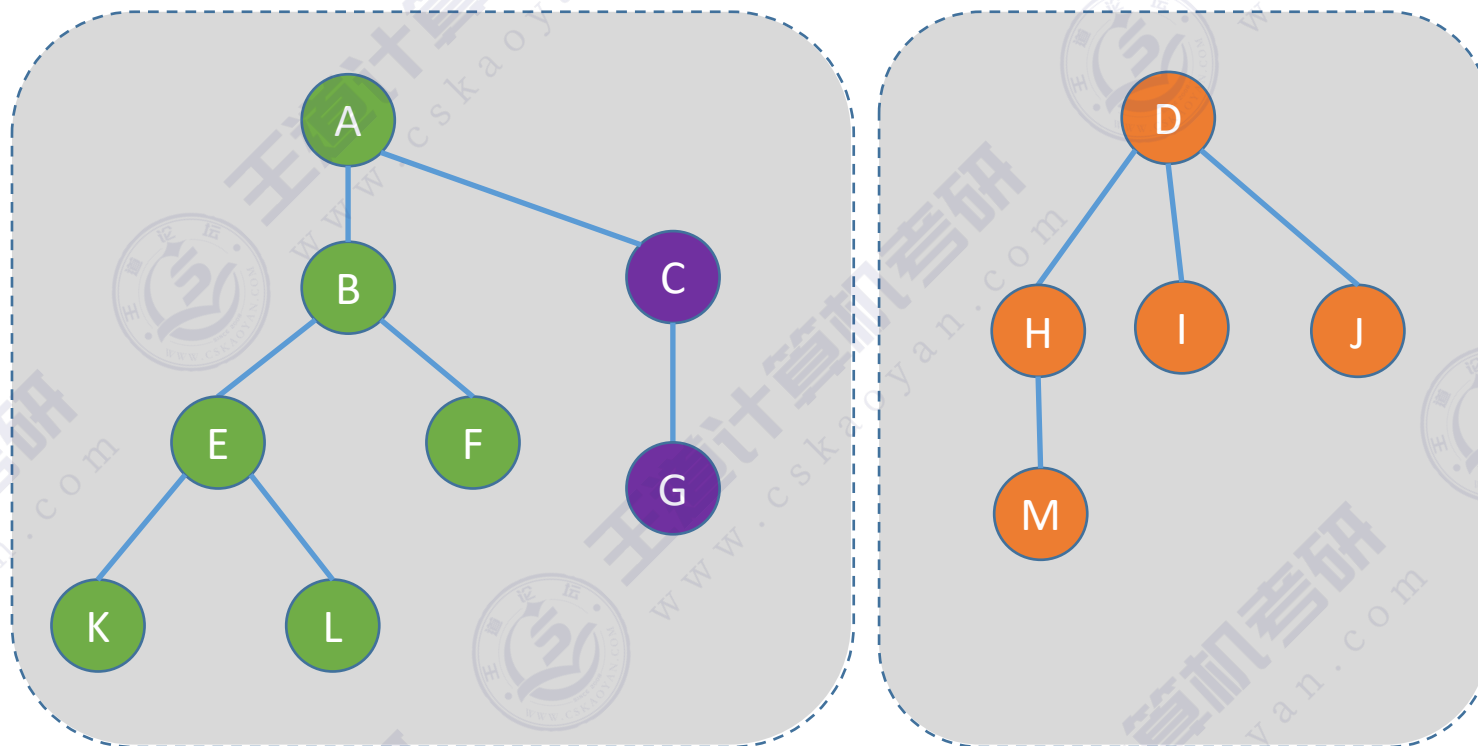
我一路向北
离开有你的季节

用互不相交的树，表示多个“集合”



如何把两个集合“并”为一个集合？

用互不相交的树，表示多个“集合”



应采用什么样的存储结构？



如何把两个集合“并”为一个集合？

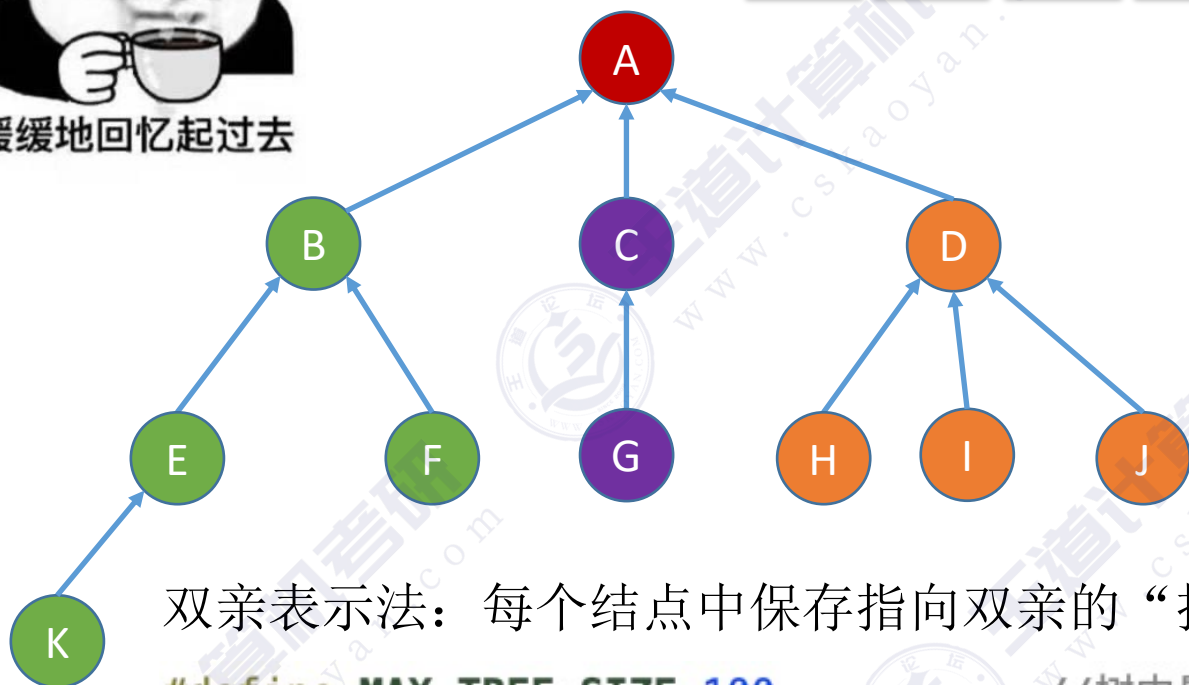
—— 让一棵树成为另一棵树的子树即可





缓缓地回忆起过去

回忆：树的存储——双亲表示法



双亲表示法：每个结点中保存指向双亲的“指针”

```
#define MAX_TREE_SIZE 100
typedef struct{
    ElemType data;
    int parent;
}PTNode;
typedef struct{
    PTNode nodes[MAX_TREE_SIZE];
    int n;
}PTree;
```

//树中最多结点数
//树的结点定义
//数据元素
//双亲位置域

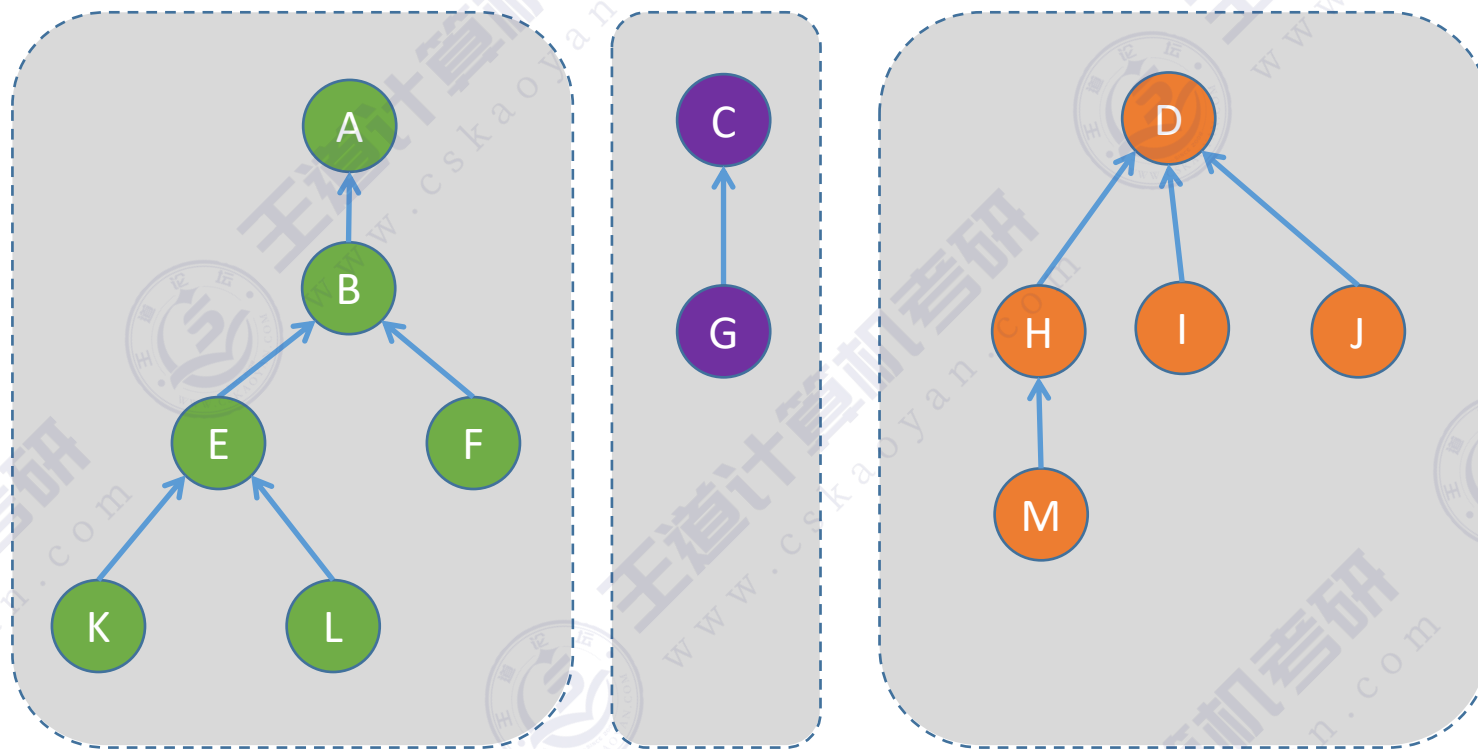
//树的类型定义
//双亲表示
//结点数

	data	parent
0	A	-1
1	B	0
2	C	0
3	D	0
4	E	1
5	F	1
6	G	2
7	H	3
8	I	3
9	J	3
10	K	4
11		
12		
13		

根节点
parent=-1

parent=4 表
示父节点的
数组下标为4

“并查集”的存储结构

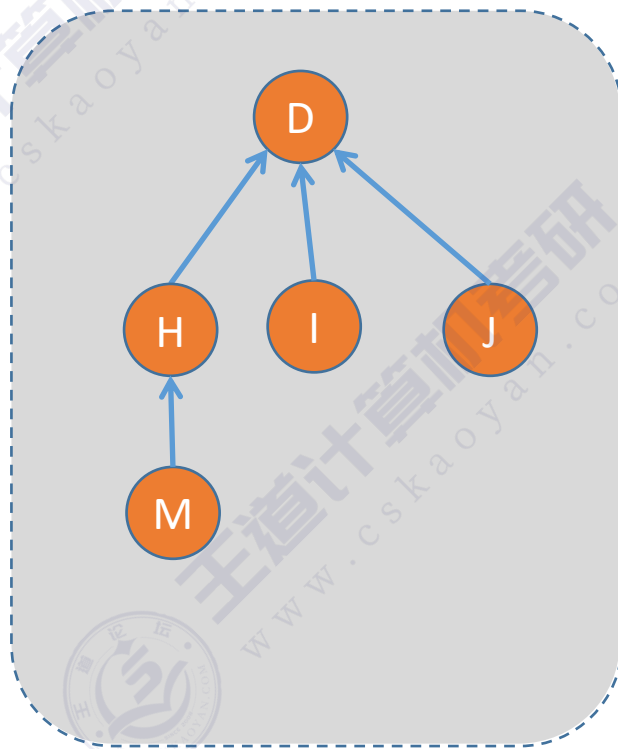
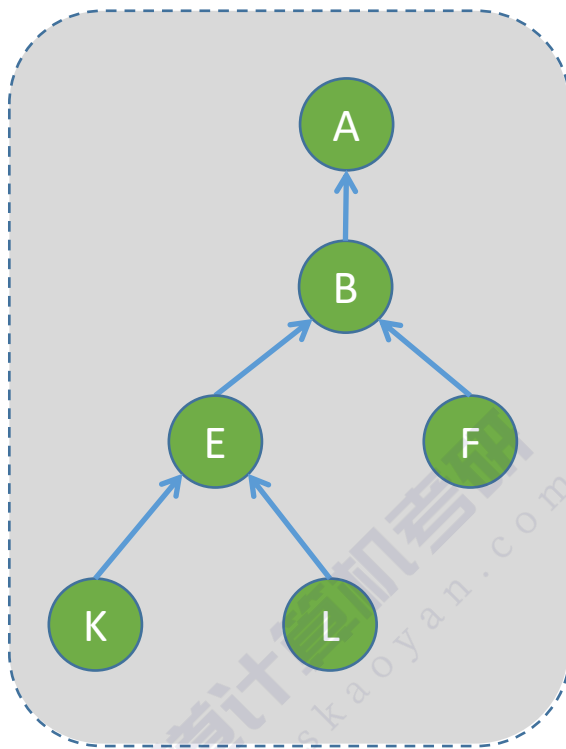


数据元素
数组下标

	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	0	-1	-1	1	1	2	3	3	3	4	4	7

用一个数组 S[] 即可表示“集合”关系

“并查集”的基本操作



集合的两个基本操作——“**并**”和“**查**”

Find —— “查”操作：确定一个指定元素所属集合

Union —— “并”操作：将两个不想交的集合合并为一个

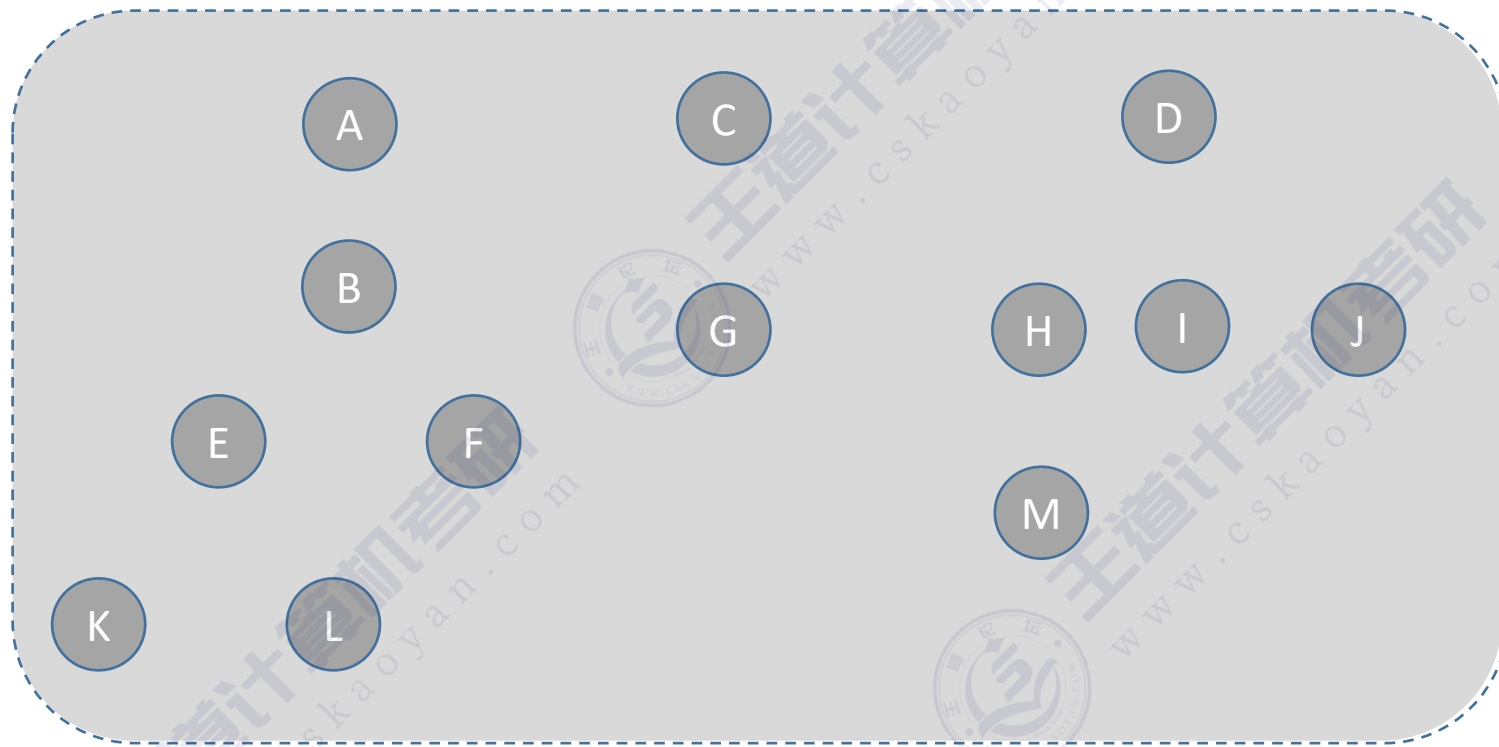
注：并查集（Disjoint Set）是逻辑结构——集合的一种具体实现，只进行“并”和“查”两种基本操作

数据元素
数组下标

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	0	-1	-1	1	1	2	3	3	3	4	4	7

用一个数组 S[] 即可表示“集合”关系

“并查集”的代码实现——初始化



```
#define SIZE 13
int UFSets[SIZE]; //集合元素数组

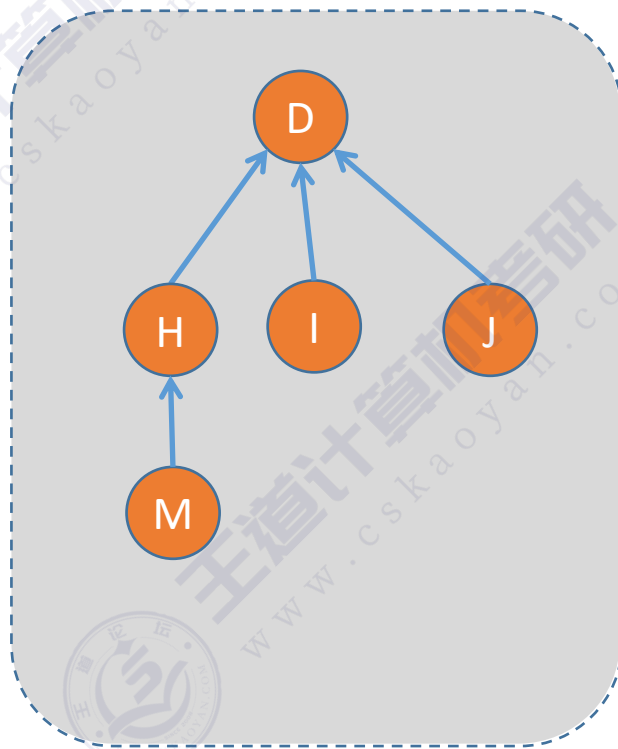
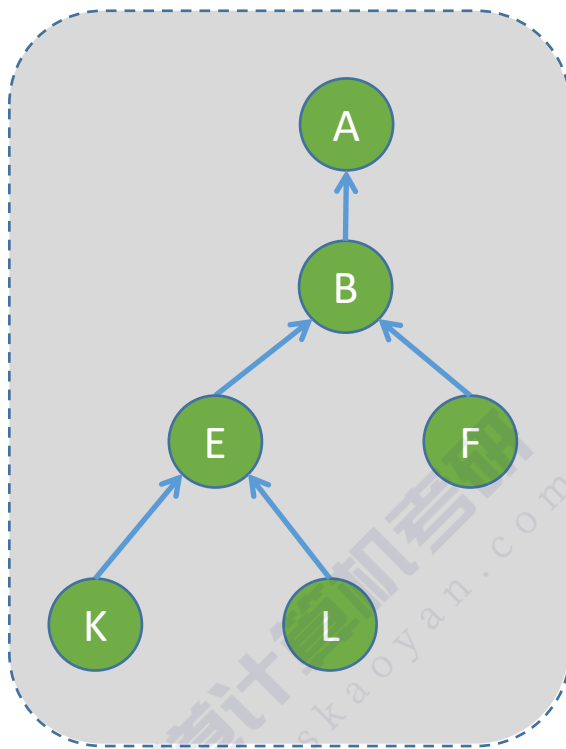
//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}
```

数据元素
数组下标

	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1	-1

用一个数组 S[] 即可表示“集合”关系

“并查集”的代码实现——并、查



```
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)           //循环寻找x的根
        x=S[x];
    return x;                //根的s[]小于0
}
```

```
//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

数据元素
数组下标

	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-1	0	-1	-1	1	1	2	3	3	3	4	4	7

用一个数组 S[] 即可表示“集合”关系

时间复杂度分析

```
#define SIZE 13
int UFsets[SIZE];    //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }

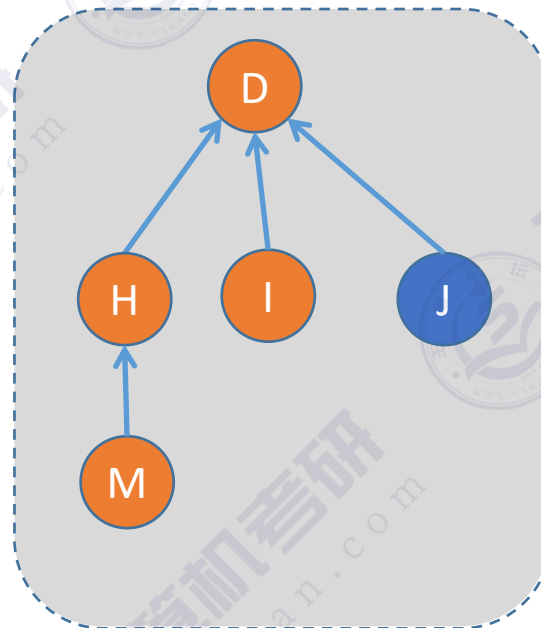
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)    //循环寻找x的根
        x=S[x];
    return x;        //根的s[]小于0
}

//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

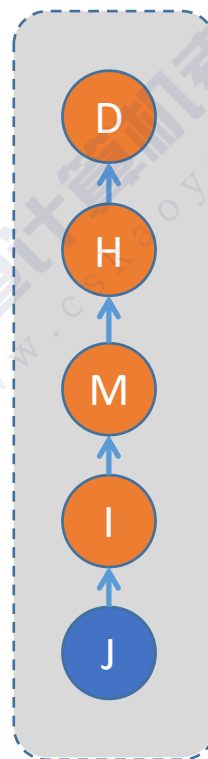
最坏时间复杂度：
 $O(n)$

时间复杂度：
 $O(1)$

找到J所属的集合



较好的情况



最坏的情况

高度
 $h=n$

若结点数为 n ，Find 最坏时间复杂度为 $O(n)$

Union 操作的优化

```
#define SIZE 13
int UFSets[SIZE];    //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }

    //Find “查”操作，找x所属集合（返回x所属根结点）
    int Find(int S[],int x){
        while(S[x]>=0)    //循环寻找x的根
            x=S[x];
        return x;        //根的s[]小于0
    }

    //Union “并”操作，将两个集合合并为一个
    void Union(int S[],int Root1,int Root2){
        //要求Root1与Root2是不同的集合
        if(Root1==Root2) return;
        //将根Root2连接到另一根Root1下面
        S[Root2]=Root1;
    }
}
```

最坏时间复杂度：
 $O(n)$

时间复杂度：
 $O(1)$

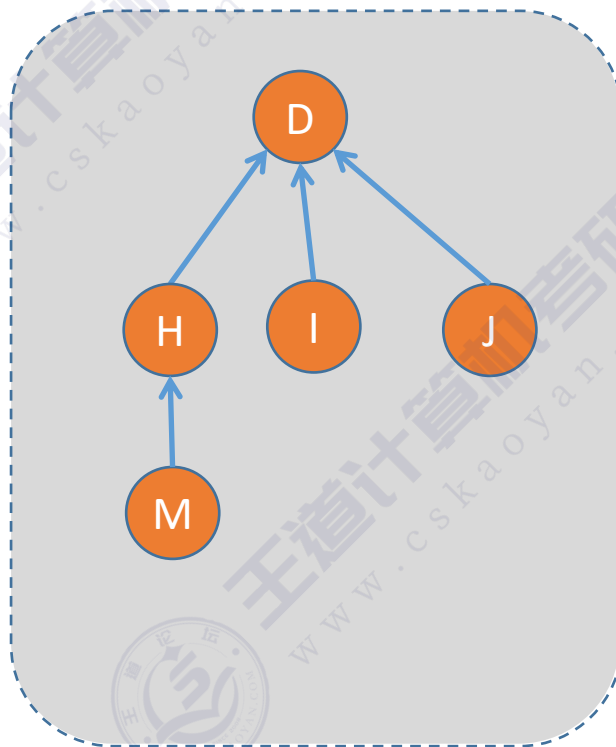
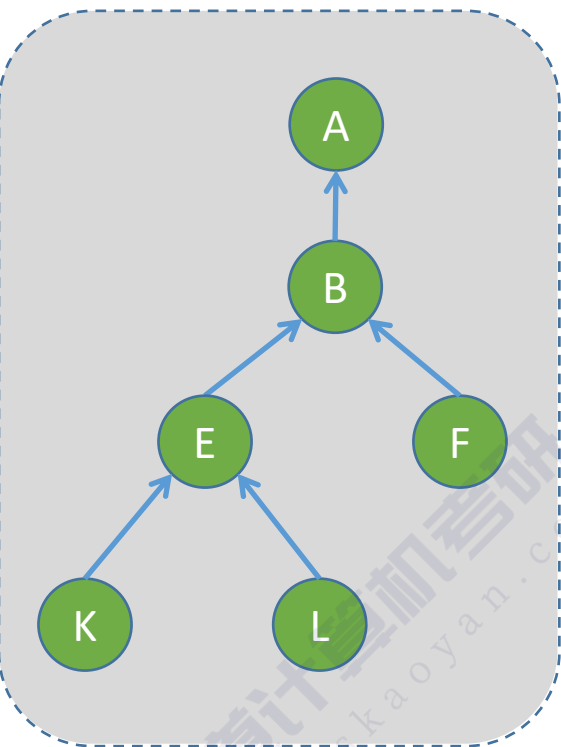
好主意



优化思路：在每次Union操作构建树的时候，尽可能让树不长高高

- ①用根节点的绝对值表示树的结点总数
- ②Union操作，让小树合并到大树

Union 操作的优化



//Union “并”操作，小树合并到大树

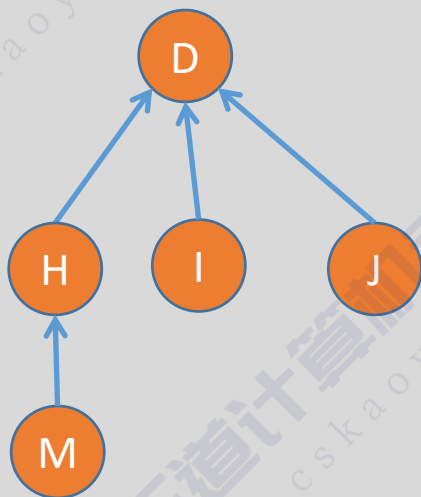
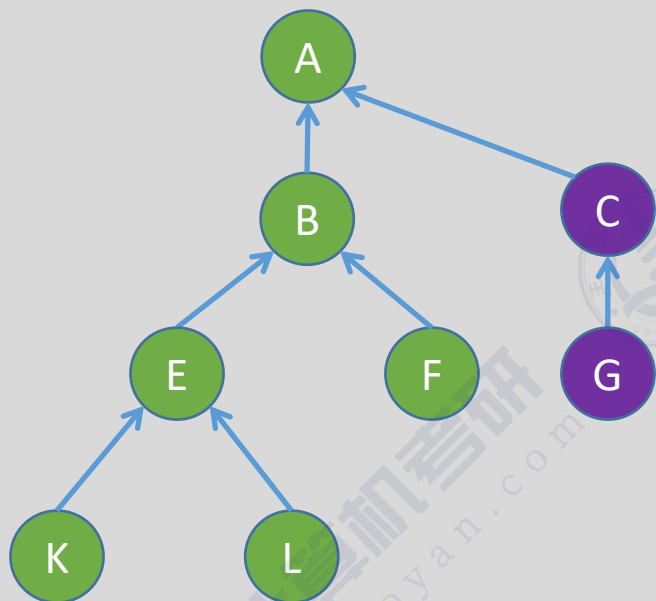
```
void Union(int S[],int Root1,int Root2){  
    if(Root1==Root2)    return;  
    if(S[Root2]>S[Root1]) { //Root2结点数更少  
        S[Root1] += S[Root2]; //累加结点总数  
        S[Root2]=Root1; //小树合并到大树  
    } else {  
        S[Root2] += S[Root1]; //累加结点总数  
        S[Root1]=Root2; //小树合并到大树  
    }  
}
```

数据元素
数组下标

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-6	0	-2	-5	1	1	2	3	3	3	4	4	7

- ①用根节点的绝对值表示树的结点总数
- ②Union操作，让小树合并到大树

Union 操作的优化



//Union “并”操作，小树合并到大树

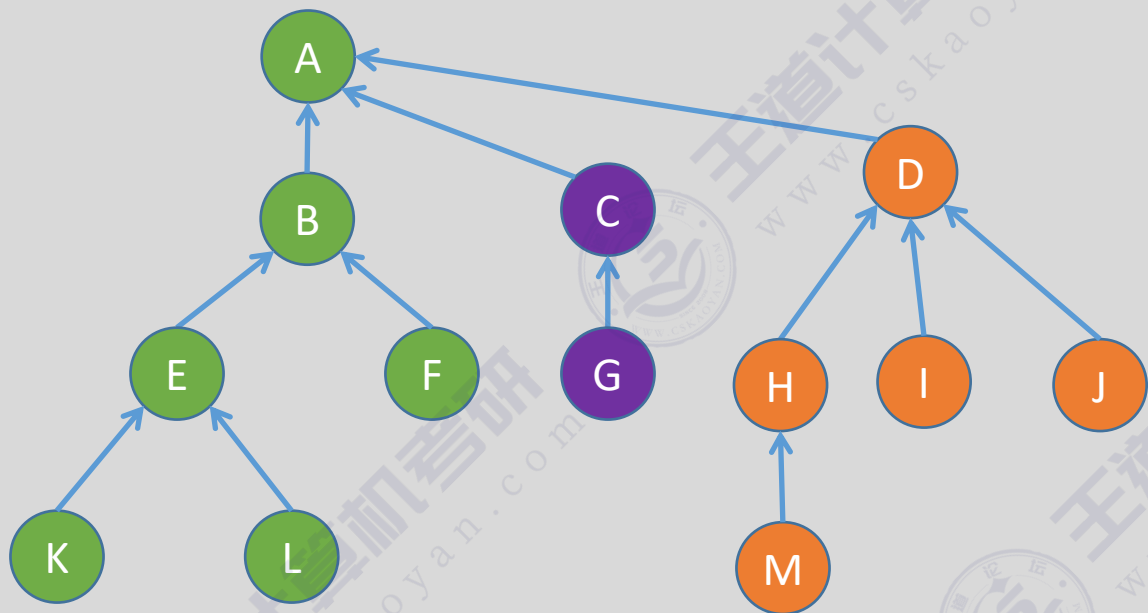
```
void Union(int S[],int Root1,int Root2){  
    if(Root1==Root2)    return;  
    if(S[Root2]>S[Root1]) { //Root2结点数更少  
        S[Root1] += S[Root2]; //累加结点总数  
        S[Root2]=Root1; //小树合并到大树  
    } else {  
        S[Root2] += S[Root1]; //累加结点总数  
        S[Root1]=Root2; //小树合并到大树  
    }  
}
```

数据元素
数组下标

	A	B	C	D	E	F	G	H	I	J	K	L	M
	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	0	-5	1	1	2	3	3	3	4	4	7

- ①用根节点的绝对值表示树的结点总数
- ②Union操作，让小树合并到大树

Union 操作的优化



//Union “并”操作，小树合并到大树

```
void Union(int S[],int Root1,int Root2){  
    if(Root1==Root2)    return;  
    if(S[Root2]>S[Root1]) { //Root2结点数更少  
        S[Root1] += S[Root2]; //累加结点总数  
        S[Root2]=Root1; //小树合并到大树  
    } else {  
        S[Root2] += S[Root1]; //累加结点总数  
        S[Root1]=Root2; //小树合并到大树  
    }  
}
```

数据元素

数组下标

S[]

A	B	C	D	E	F	G	H	I	J	K	L	M
0	1	2	3	4	5	6	7	8	9	10	11	12

-13	0	0	0	1	1	2	3	3	3	4	4	7
-----	---	---	---	---	---	---	---	---	---	---	---	---

①用根节点的绝对值表示树的结点总数

②Union操作，让小树合并到大树

Union 操作的优化

```
#define SIZE 13
int UFSets[SIZE];    //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }

    //Find “查”操作，找x所属集合（返回x所属根结点）
    int Find(int S[],int x){
        while(S[x]>=0)    //循环寻找x的根
            x=S[x];
        return x;        //根的s[]小于0
    }
}
```

Union操作优化后，
Find 操作最坏时间
复杂度： $O(\log_2 n)$

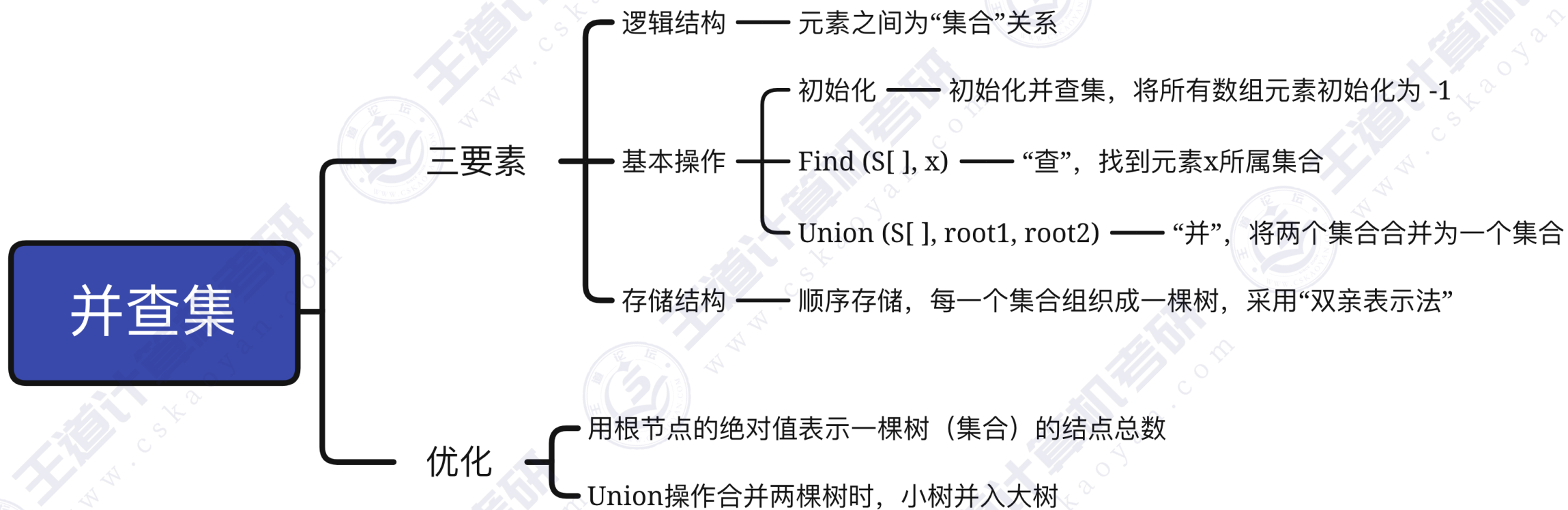
该方法构造的树高不超过 $\lfloor \log_2 n \rfloor + 1$

```
//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

优化

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]) { //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```

知识回顾与重要考点



$$\text{树高} \leq \lfloor \log_2 n \rfloor + 1$$

$$\text{Find} \rightarrow O(\log_2 n)$$

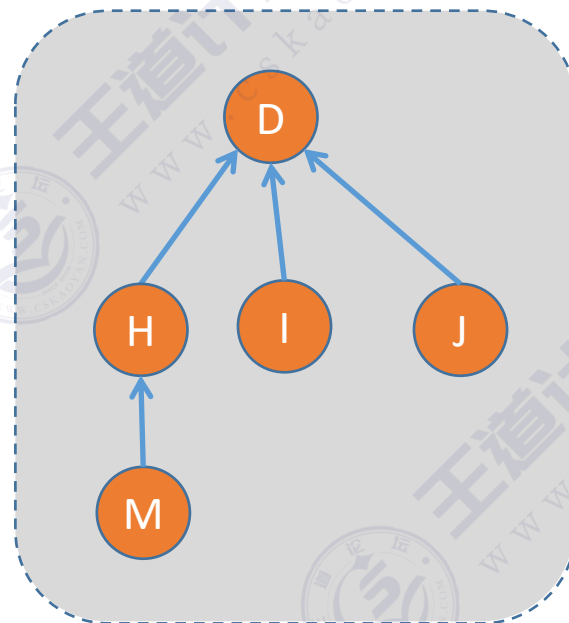
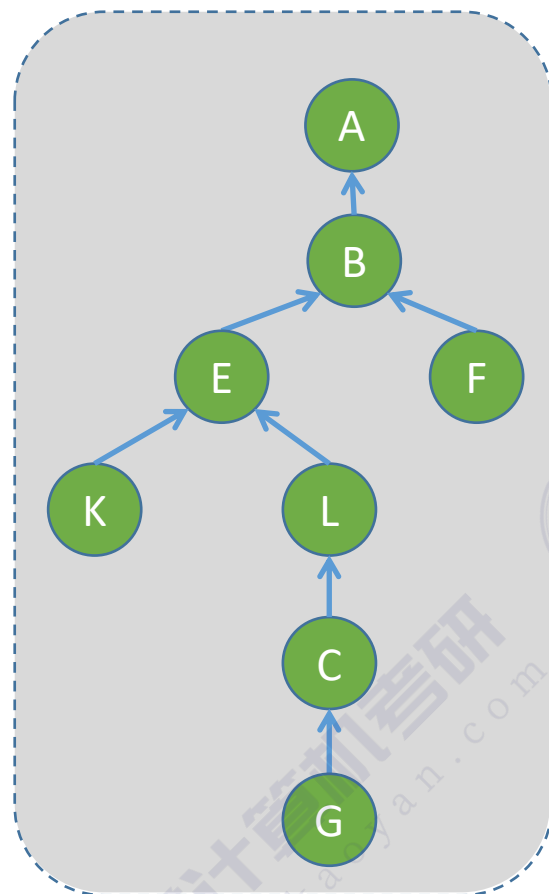
本节内容

补充：并查集的 终极优化



年轻人
不讲武德

拓展：Find 操作的优化（压缩路径）



```

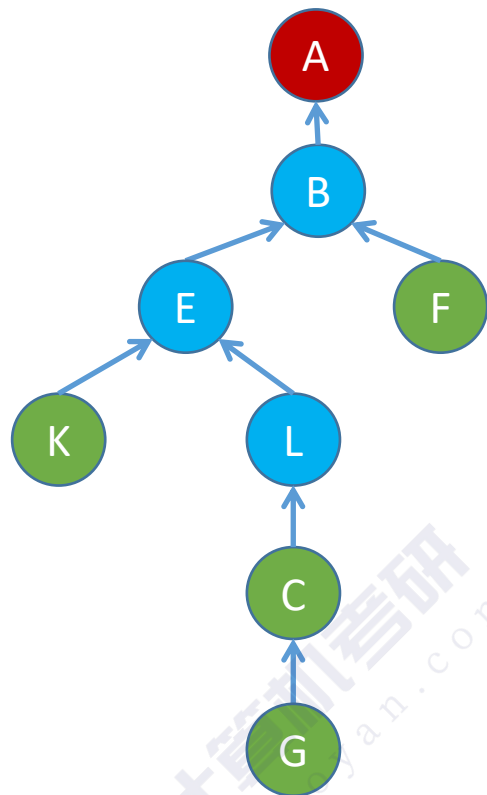
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)           //循环寻找x的根
        x=S[x];
    return x;               //根的s[]小于0
}
    
```

压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	1	1	2	3	3	3	4	4	7

Eg: Find(S[], 11) //查找结点L所属集合

拓展：Find 操作的优化（压缩路径）

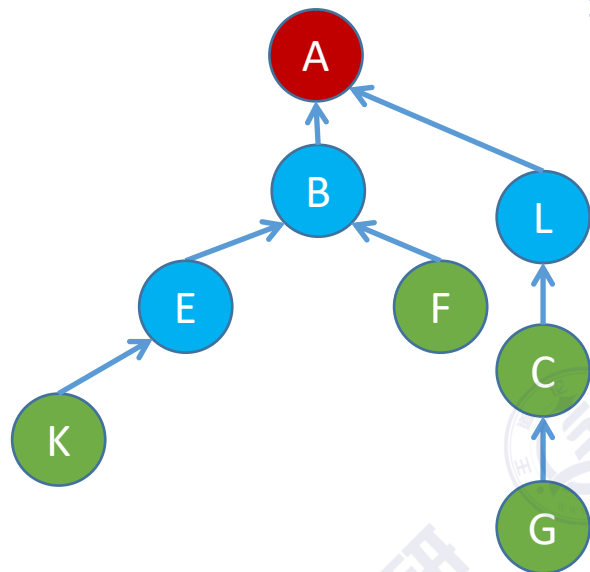


压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	1	1	2	3	3	3	4	4	7

Eg: Find(S[], 11) //查找结点L所属集合

拓展：Find 操作的优化（压缩路径）

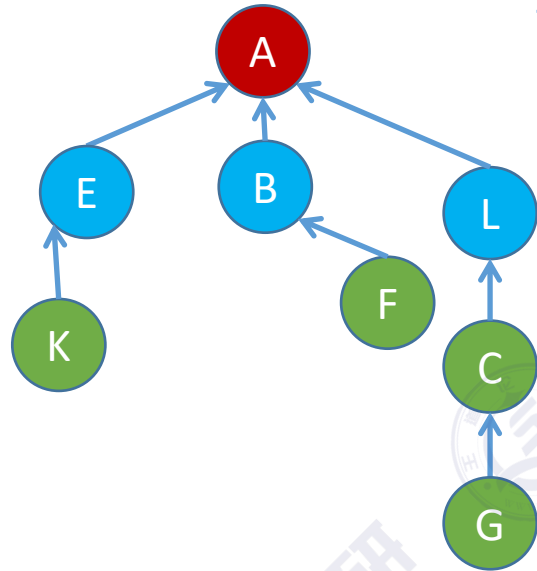


压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	1	1	2	3	3	3	4	0	7

Eg: Find(S[], 11) //查找结点L所属集合

拓展：Find 操作的优化（压缩路径）

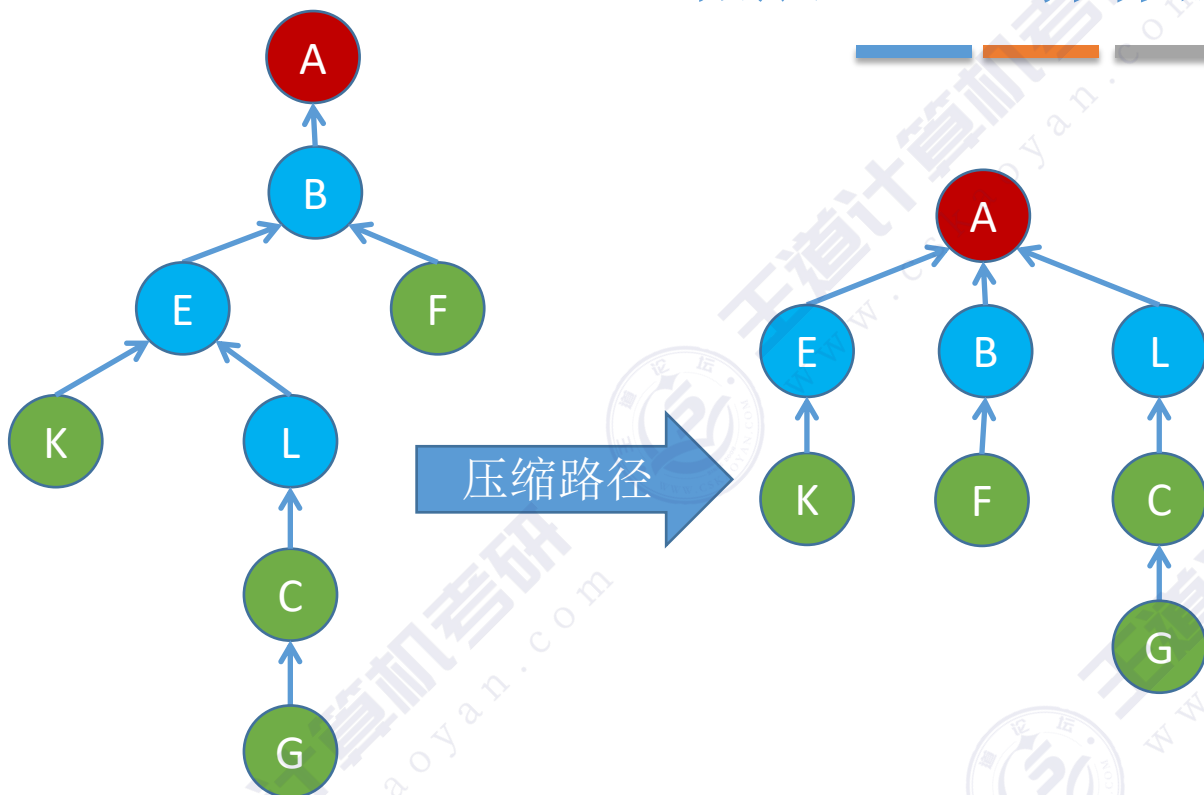


压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	0	1	2	3	3	3	4	0	7

Eg: Find(S[], 11) //查找结点L所属集合

拓展：Find 操作的优化（压缩路径）



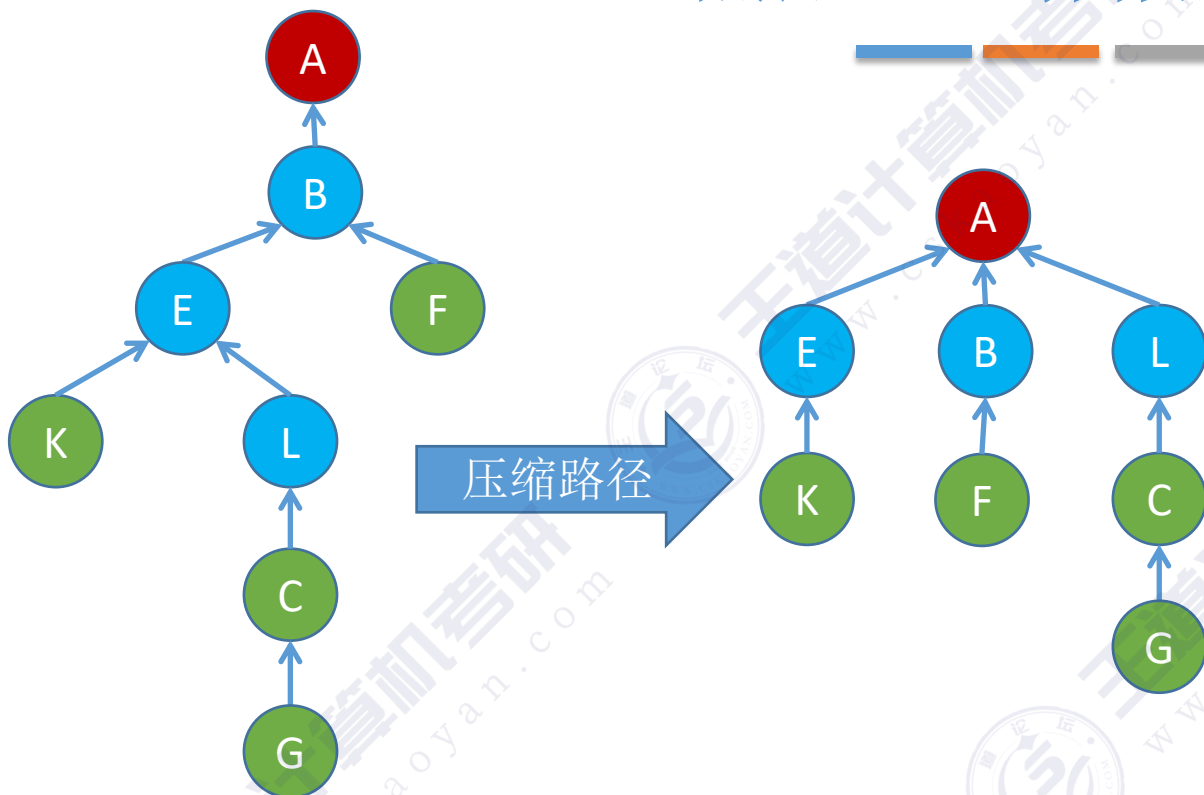
```
//Find “查”操作优化，先找到根节点，再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0) root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x]; //t指向x的父节点
        S[x]=root; //x直接挂到根节点下
        x=t;
    }
    return root; //返回根节点编号
}
```

压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

数据元素	A	B	C	D	E	F	G	H	I	J	K	L	M
数组下标	0	1	2	3	4	5	6	7	8	9	10	11	12
S[]	-8	0	11	-5	0	1	2	3	3	3	4	0	7

Eg: Find(S[], 11) //查找结点L所属集合

拓展：Find 操作的优化（压缩路径）



```
//Find “查”操作优化，先找到根节点，再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0) root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x]; //t指向x的父节点
        S[x]=root; //x直接挂到根节点下
        x=t;
    }
    return root; //返回根节点编号
}
```

压缩路径——Find 操作，先找到根节点，再将查找路径上所有结点都挂到根结点下

每次 Find 操作，先找根，再“压缩路径”，可使树的高度不超过 $O(\alpha(n))$ 。 $\alpha(n)$ 是一个增长很缓慢的函数，对于常见的 n 值，通常 $\alpha(n) \leq 4$ ，因此优化后并查集的Find、Union操作时间开销都很低

并查集的优化

```
#define SIZE 13
int UFsets[SIZE];    //集合元素数组

//初始化并查集
void Initial(int S[]){
    for(int i=0;i<SIZE;i++){
        S[i]=-1;
    }
}
```

```
//Find “查”操作，找x所属集合（返回x所属根结点）
int Find(int S[],int x){
    while(S[x]>=0)    //循环寻找x的根
        x=S[x];
    return x;        //根的s[]小于0
}
```

```
//Union “并”操作，将两个集合合并为一个
void Union(int S[],int Root1,int Root2){
    //要求Root1与Root2是不同的集合
    if(Root1==Root2) return;
    //将根Root2连接到另一根Root1下面
    S[Root2]=Root1;
}
```

核心思想：尽可能让树变矮

Find优化
(压缩路径)

Union优化

```
//Find “查”操作优化，先找到根节点，再进行“压缩路径”
int Find(int S[],int x){
    int root = x;
    while(S[root]>=0) root=S[root]; //循环找到根
    while(x!=root){ //压缩路径
        int t=S[x]; //t指向x的父节点
        S[x]=root; //x直接挂到根节点下
        x=t;
    }
    return root; //返回根节点编号
}
```

```
//Union “并”操作，小树合并到大树
void Union(int S[],int Root1,int Root2){
    if(Root1==Root2) return;
    if(S[Root2]>S[Root1]){ //Root2结点数更少
        S[Root1] += S[Root2]; //累加结点总数
        S[Root2]=Root1; //小树合并到大树
    } else {
        S[Root2] += S[Root1]; //累加结点总数
        S[Root1]=Root2; //小树合并到大树
    }
}
```


并查集的优化

用数组、双亲表示法来描述元素之间的集合关系。根节点为 -1，非根节点指向父节点下标。

Find (int S[], x) ——找到x所属集合

Union(int S[], int x, int y) —— 将x、y 所属集合合并

最坏时间复杂度:

Find 操作 = 最坏树高 = $O(n)$

将n个独立元素通过多次Union 合并为一个集合—— $O(n^2)$

Union
优化

用根节点的负值表示
一棵树的结点总数

每次 Union 操作让 **小树合并到大树** 根节点下面

最坏时间复杂度:

Find 操作=最坏树高= $O(\log_2 n)$

将n个独立元素通过多次Union 合并为一个集合—— $O(n \log_2 n)$

Find
优化

“**压缩路径**”的策略，每次 Find 操作先找到 x 所属根节点，再将查找路径上的所有结点都直接挂在根节点下面

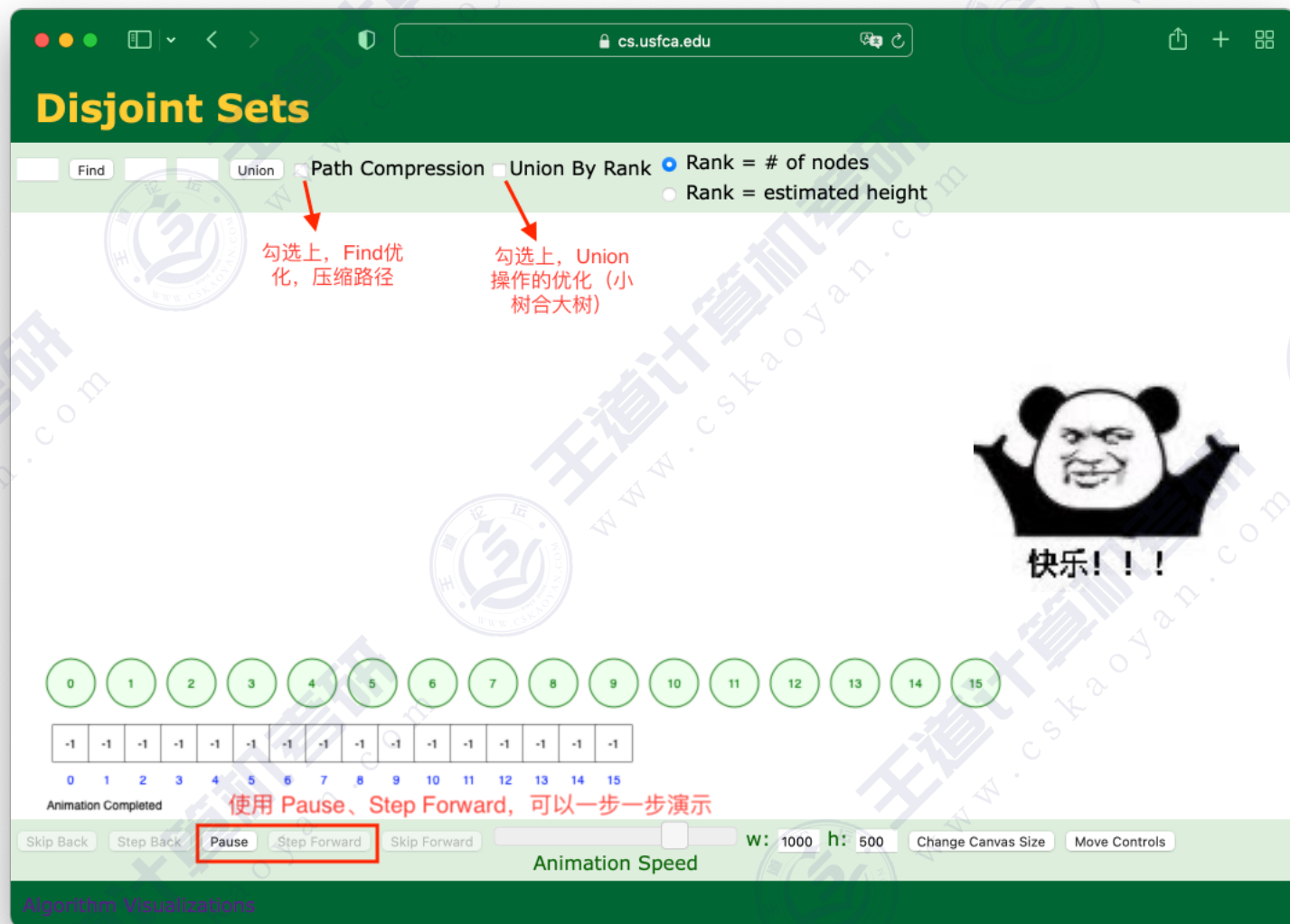
最坏时间复杂度:

Find 操作=最坏树高= $O(\alpha(n))$

将n个独立元素通过多次Union 合并为一个集合—— $O(n \alpha(n))$

408快乐站

点开链接玩一玩: <https://www.cs.usfca.edu/~galles/visualization/DisjointSets.html>



玩玩各种算法: <https://www.cs.usfca.edu/~galles/visualization/Algorithms.html>

David Galles
Computer Science
University of San
Francisco

- Reversing a String
- N-Queens Problem
- Indexing
 - Binary and Linear Search (of sorted list) → 二分查找 和 顺序查找
 - Binary Search Trees → 二叉查找树BST
 - AVL Trees (Balanced binary search trees) → 平衡二叉树AVL
 - Red-Black Trees → 红黑树
 - Splay Trees
 - Open Hash Tables (Closed Addressing) → 散列表 (拉链法解决冲突)
 - Closed Hash Tables (Open Addressing) → 散列表 (开放定址法解决冲突)
 - Closed Hash Tables, using buckets
 - Trie (Prefix Tree, 26-ary Tree)
 - Radix Tree (Compact Trie)
 - Ternary Search Tree (Trie with BST of children)
 - B Trees
 - B+ Trees → B树、B+树 (注: 心中有B树就行, 408不要求掌握B+树的插入删除)
- Sorting
 - Comparison Sorting
 - Bubble Sort → 冒泡排序
 - Selection Sort → 选择排序
 - Insertion Sort → 插入排序
 - Shell Sort → 希尔排序
 - Merge Sort → 归并排序
 - Quick Sort → 快速排序
 - Bucket Sort
 - Counting Sort
 - Radix Sort → 基数排序 (该网站实现方式和教材中讲授方式不同)
 - Heap Sort → 堆排序
- Heap-like Data Structures
 - Heaps
 - Binomial Queues
 - Fibonacci Heaps
 - Leftist Heaps
 - Skew Heaps
- Graph Algorithms
 - Breadth-First Search → 图的广度优先遍历
 - Depth-First Search → 图的深度优先遍历
 - Connected Components
 - Dijkstra's Shortest Path → 迪杰斯特拉算法
 - Prim's Minimum Cost Spanning Tree → Prim算法求MST
 - Topological Sort (Using Indegree array) → 拓扑排序算法
 - Topological Sort (Using DFS)
 - Floyd-Warshall (all pairs shortest paths) → Floyd算法
 - Kruskal Minimum Cost Spanning Tree Algorithm → Kruskal算法
- Dynamic Programming
 - Calculating nth Fibonacci number
 - Making Change
 - Longest Common Subsequence
- Geometric Algorithms
 - 2D Rotation and Scale Matrices
 - 2D Rotation and Translation Matrices
 - 2D Changing Coordinate Systems
 - 3D Rotation and Scale Matrices
 - 3D Changing Coordinate Systems
- Others ...
 - Disjoint Sets → 并查集
 - Huffman Coding (available in java version)



快乐!!!

欢迎大家对本节视频进行评价~



学员评分：5.5.2_2 并...

扫一扫二维码打开或分享给好友



— 腾讯文档 —

可多人实时在线编辑，权限安全可控



公众号：王道在线



b站：王道计算机教育



抖音：王道计算机考研